

# SHOULD HAVE A BETTER TITLE

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

We proposed a novel paradigm for implementing latent reasoning by combining DAPO-like reinforcement learning and special gating units. In this way, we transformed text-based explicit reasoning models to latent reasoning models, progressively and practically. Initial experiments demonstrate its feasibility, with significant stability and the least extra training overhead by parallelism and minimal changes, while holding full potential to scale up.

## 1 INTRODUCTION

*Waiting for rewrite.*

## 2 RELATED WORK

**Explicit Reasoning Models and Large-scale Reinforcement Learning** Advances in explicit reasoning models, including OpenAI o1 & o3 (OpenAI, 2025; 2024) and DeepSeek R1 (DeepSeek-AI, 2025), demonstrated their strong ability in mathematical reasoning. Based on Chain-of-Thought (Wei et al., 2022), models significantly improved arithmetic, commonsense, and symbolic reasoning tasks with explicit intermediate reasoning text.

Many paradigms were also created from explicit reasoning models subsequently, including Tree-of-Thought (Yao et al., 2023), Graph-of-Thought (Besta et al., 2024), which are also proposed to improve the reasoning ability of LLMs.

Ouyang et al. (2022) showed how GPT-3 (Kojima et al., 2023) can be fine-tuned with reinforcement learning via Proximal Policy Optimization (PPO). PPO is a reinforcement learning algorithm proposed by Schulman et al. (2017), which optimizes the agent’s policy by maximizing the expected reward while ensuring the new policy does not deviate excessively from the old policy. Shao et al. (2024) Proposed Group Relative Policy Optimization (GRPO), which overcame the limitations of PPO, including the need for a large number of samples and the difficulty of tuning hyperparameters. Yu et al. (2025) Demonstrated the effectiveness of Decoupled Clip and Dynamic Sampling Policy Optimization (DAPO) in large-scale reinforcement learning, which performed best in several benchmarks on Qwen2.5-32B Yang et al. (2024b) solely based on pure reinforcement learning and the base instruction model of Qwen.

In contrast, our work leverages a latent space instead of explicit reasoning text, which reduces verbosity and improves efficiency. Based on large-scale reinforcement learning, we effectively transplant those explicit reasoning-based optimization and advances into the latent reasoning region.

**Latent Space Reasoning** Unlike explicit reasoning, latent reasoning paradigms leverage latent spaces, promoting efficiency and accuracy. Previous works have already found that LLMs may involve hidden computation steps, which are not explicitly generated in text (Yang et al., 2024c; Chen et al., 2023). Driven by this spirit, Hao et al. (2024) proposed COCONUT, assigning models to “reason in a latent continuous space” through specifically designed training algorithms, and demonstrated its potential compared to the explicit Chain-of-Thought reasoning procedure, especially on tasks requiring backtracking.

Geiping et al. (2025) Introduced a new recurrent transformer architecture that lets models perform multiple hidden reasoning iterations per token, as they suggested that “a recurrent block can be looped arbitrarily at inference to refine the hidden state before producing the next token.” Chen

et al. (2025) argued that LLMs may have an inherent potential to reason internally, suggesting that LLMs can learn better reasoning policies in the latent space when given appropriate training signals. Saunshi et al. (2025) Demonstrated that the critical impact of model capabilities lies in its depth instead of parameter size. In this way, they proposed *Looped Transformer*, which generated latent thoughts and simulated Chain-of-Thought reasoning procedures internally. They also pretrained a 3.5B model that might compete with 50B ones.

In contrast, our work focuses on the practical implementation of latent reasoning, instead of cold-start training, which is done by Saunshi et al.. Our paradigm in feasibility and stability stands in the approach to training the model without specifically designed trajectories, e.g., stepping guidance, from a production-level model via small injection with VAE and gates. It also combines the advances of explicit reasoning models and large-scale reinforcement learning objectives. At the same time, prior work mainly focused on producing the latent reasoning space, and small-scale usages primarily based on SFT.

### 3 METHODOLOGY

We first introduce the general structure of modern large language models (LLMs) to demonstrate our approach and paradigm better.

Modern LLMs involve the Embedding layer; Core Transformer, including Self-Attention and FFN; and the Output linear layer which projects the latent state vector of `embed_dim` to probability distribution of `vocab_size` layers, as we denote them  $E$ ,  $M_n$  and  $O$  respectively, which  $n$  indicates the layer of hidden state (Vaswani et al., 2017; Radford et al., 2019).

#### 3.1 MODEL ARCHITECTURE

*Waiting for writing.*

#### 3.2 INFERENCE

We used middle layers to enter the latent reasoning space to avoid breaking special usage in final layers (Saunshi et al., 2025).

The whole inference procedure is described in (...)

We preliminarily perform prefilling by the question with the length  $q_0$  into the model and get the prefilled KV Cache (Pope et al., 2022) and the probability distribution of the answer’s first token  $t_{q_0+1}$  (starting to count from  $t_1$ ).

After extracting the hidden state  $\mathbf{h}_{n_0, q_0}$  (where  $n_0$  denotes the layer number of the Transformer and  $q_0$  denotes question length), we deliver the state to a Variational Autoencoder (Kingma and Welling, 2019) (we will introduce it in 3.2.1), and produce a vector for storage with length  $l_s$ .

##### 3.2.1 VARIATIONAL AUTOENCODER LAYER

We use Variational Autoencoder (VAE) (Kingma and Welling, 2019) to compress and uncompress the hidden state token  $\mathbf{h}_{n,k}$ , which is an  $l_h$ -dimensional vector, to an  $l_c$ -dimensional vector and then back to  $l_h$ .

The VAE layer looks like a bottleneck-like layer, which is used to compress the hidden state to a lower dimension and then expand it back to the original size. The shape of the VAE layer in our work can be described in Figure 1.

We pre-trained the VAE on the online-generated activation of 1024 consequences, and the validation loss decreased steadily, proving that the VAE’s pre-training process requires little data. We will provide supplemental data demonstrating its effectiveness in the appendix.

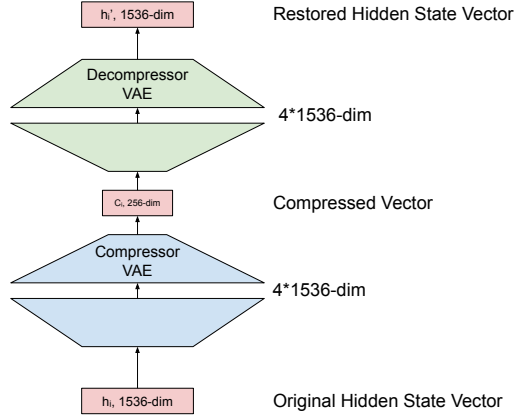


Figure 1: The VAE layer. We first compress the hidden state to a lower dimension (expand into 4 times the original size and then compress), extracting the compressed vector, and use the symmetric approach to expand it back to the original dimension.

**Long-term Cache** We will store the latent space activation output from later  $n_0$  for further parallel training. Yet directly storing the raw vector of embed\_dim could be space-consuming. (see <sup>1</sup> for quantitative analysis of the space consumption),

We store it in each operation and concatenate these vectors into a matrix subsequently as a hidden state.

### 3.2.2 GATING MECHANISM

Our model introduces the update gate and the Dynamic Current Step Mixing Gate.

1. **Update Gate.** Inspired by Gated Recurrent Unit (GRU) (Zhang et al., 2023; Chung et al., 2014; Cho et al., 2014), the update gate is used to increase its contextual consistency in latent space, but the value of the gate is derived from the sequence. We denote it as  $g_t$ , and its derivation procedure can be described in Equation (not yet). We initialize it to all zeros, i.e., initially the model is equivalent to the original one, but it evolves as  $g$  moves away from 0.
2. **Dynamic Current-Step-Mixing Gate (DMG).** The injection ratio of the latent state vector needs to vary depending on the next token, yet directly applying one fixed gate for all cannot fit this need. Therefore, we add a DMG to gate the Update Gate according to both the latent state vector and the embedding of the next token.

The processed vector  $h'_{n,k}$  after the VAE layer is passed to several gates, as we denote in Equation 1.

$$h'_{n,k} = h'_{n,k-1} \odot g \odot \text{DMG}(e_k) \times s_{\text{inj}} + (1 - g) \odot e_k \quad (1)$$

Where  $e_k = \text{Embedding}(t_k)$ ;  $t_k$  denotes the token of the sequence with position  $k$ , and  $s_{\text{inj}}$  denotes the injection scalar, which is a constant scalar to bridge the difference in order of magnitude between the gating value and the result of Dynamic Current-Step-Mixing Gate, which will be described in Section 3.2.2.

<sup>1</sup>Consider a sample batch size of 1024, sequence length 1024, and embedding dimension 1536. Each element occupies 2 bytes (BF16), resulting in  $1024 \times 1024 \times 1536 \times 2 = 2^{31.5}$  bytes, or approximately 2.8 GB of VRAM. In contrast, if we adapt the VAE layer to compress the hidden state to 256 dimensions, we only need  $1024 \times 1024 \times 256 \times 2 = 2^{29}$  bytes, or approximately 512 MB of VRAM.

### 3.3 TRAINING

It is hard for us to train models in a budget-tight scenario, so we adapted Low-Rank Adaptation (LoRA) (Hu et al., 2021) to our model.

We use the DAPO algorithm as our reinforcement learning framework, but the whole procedure differs slightly from the original DAPO. The whole training procedure is described in Algorithm 1.

---

**Algorithm 1:** Training Procedure taking one batch as an example.

---

**Data:** Given a sequence  $S$  of seq\_length, where  $t_k \in S$ ;  $t_k$  denotes token index.

**Result:** Model

```

1  $e \leftarrow \text{Embedding}(s[: - 1]);$ 
2  $h \leftarrow E(e);$ 
3  $p \leftarrow \text{Softmax}(O(h));$ 
4  $loss \leftarrow \text{CrossEntropyLoss}(p, \text{one\_hot}([: - 1]));$ 
5 Optimize by minimizing  $loss$ ;
```

---

#### 3.3.1 REINFORCEMENT LEARNING

We parallelized several inference procedures simultaneously, preferably 16 for the same question, and verified the answer (Kydliček).

**Training Objective** We adapt the DAPO (Yu et al., 2025) algorithm, but we have not implemented the clip mechanism, which is used to clip the reward and penalty to a specific range. Hence, we currently use the original DAPO algorithm, which is shown in Equation 2.

$$\begin{aligned}
\mathcal{J}_{\text{DAPO}}(\theta) = & \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \\
& \frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} r_{i,t}(\theta) \hat{A}_{i,t} \\
\text{subject to } & 0 < |\{o_i \mid \text{is\_equivalent}(a, o_i)\}| < G,
\end{aligned} \tag{2}$$

Where

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}, \quad \hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^G)}{\text{std}(\{R_i\}_{i=1}^G)}. \tag{3}$$

**Award Mechanism** We use reward<sub>acc</sub> and penalty<sub>len</sub> to award or penalize in accuracy, according to DAPO (Yu et al., 2025). The exact value of the reward and penalty will be described in the appendices.

**Positive-Negative Clipping** We choose only all the positive samples to eliminate the issue of too many negative samples (each of which is less useful for improving the model) under a budget-tight scenario. We randomly choose the remaining negative samples according to the ratio 1 : 1.

#### 3.3.2 TIME CONSISTENCY REGULARIZATION

We adopt a mechanism called ‘‘Gating Value Bonus,’’ which is used to let the model make full use of the information from the latent space instead of getting stuck in the local optima of not using the space sufficiently.

The gating value bonus is a regularization term designed to constrain gating values toward the mid-point, as the formulation is shown in Equation 4.

$$\mathcal{J}_{\text{gate}}(g) = \frac{1}{\text{output\_dim}} C_0 \sum_{i=0}^{\text{output\_dim}} \exp \left[ \lambda \left( \frac{1}{2} - g_i \right)^2 \right] \tag{4}$$

Where  $g_i = \mathbf{g} \cdot \mathbf{e}^{(i)}$ , i.e., the  $i$ -th element of the gate vector  $\mathbf{g}$ , and  $C_0$  is a constant to control the scale of the bonus;  $\lambda$  is a constant to control the rate of convergence.

### 3.3.3 PARALLELIZED TRAINING

After sampling, we concatenate the question, answer, and hidden state and apply them to the training procedure.

## 4 EXPERIMENTS

### 4.1 SETUP

**Base Model** We use Qwen3-4B (Yang et al., 2024a;b; Team, 2025) as our base model, which is a 4B-parameter model with 24 layers. DeepSeek distilled the model into an explicit reasoning model (DeepSeek-AI, 2025), which is our comparison baseline.

**Baselines** We compare our model with the following baselines:

- **Qwen3-4B:** The original model.
- Distillation method from DeepSeek-AI (2025) performed on Qwen3-4B.
- The original model with DAPO (Yu et al., 2025) objective solely.
- Our model with DAPO algorithm, gating mechanism, and VAE layer.
- The original model with VAPO (Yue et al., 2025) objective solely.

## 5 DISCUSSION

After conducting several experiments and analyzing the theoretical bases, we have several discussions:

## 6 CONCLUSION

## REFERENCES

- M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefler. Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690, Mar. 2024. ISSN 2159-5399. doi: 10.1609/aaai.v38i16.29720. URL <http://dx.doi.org/10.1609/aaai.v38i16.29720>.
- H. Chen, Y. Feng, Z. Liu, W. Yao, A. Prabhakar, S. Heinecke, R. Ho, P. L. Mui, S. Savarese, C. Xiong, and H. Wang. Language models are hidden reasoners: Unlocking latent reasoning capabilities via self-rewarding, 2025. URL <https://openreview.net/forum?id=4Po8d9GAfQ>.
- W. Chen, M. Yin, M. Ku, P. Lu, Y. Wan, X. Ma, J. Xu, X. Wang, and T. Xia. Theoremqa: A theorem-driven question answering dataset, 2023. URL <https://arxiv.org/abs/2305.12524>.
- K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio. On the properties of neural machine translation: Encoder-decoder approaches. *CoRR*, abs/1409.1259, 2014. URL <http://arxiv.org/abs/1409.1259>.
- J. Chung, Ç. Gülçehre, K. Cho, and Y. Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014. URL <http://arxiv.org/abs/1412.3555>.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.

- J. Geiping, S. McLeish, N. Jain, J. Kirchenbauer, S. Singh, B. R. Bartoldson, B. Kailkhura, A. Bhatele, and T. Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach, 2025. URL <https://arxiv.org/abs/2502.05171>.
- S. Hao, S. Sukhbaatar, D. Su, X. Li, Z. Hu, J. Weston, and Y. Tian. Training large language models to reason in a continuous latent space, 2024. URL <https://arxiv.org/abs/2412.06769>.
- E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, and W. Chen. Lora: Low-rank adaptation of large language models. *CoRR*, abs/2106.09685, 2021. URL <https://arxiv.org/abs/2106.09685>.
- D. P. Kingma and M. Welling. An introduction to variational autoencoders. *CoRR*, abs/1906.02691, 2019. URL <http://arxiv.org/abs/1906.02691>.
- T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa. Large language models are zero-shot reasoners, 2023. URL <https://arxiv.org/abs/2205.11916>.
- H. Kydlíček. Math-Verify: Math Verification Library. URL <https://github.com/huggingface/math-verify>.
- OpenAI. Introducing openai o1, September 2024. URL <https://openai.com/o1/>. Accessed: 2025-04-27.
- OpenAI. Introducing openai o3 and o4-mini, April 2025. URL <https://openai.com/index/introducing-o3-and-o4-mini/>. Accessed: 2025-04-27.
- L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback, 2022. URL <https://arxiv.org/abs/2203.02155>.
- R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean. Efficiently scaling transformer inference, 2022. URL <https://arxiv.org/abs/2211.05102>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL [https://cdn.openai.com/better-language-models/language\\_models\\_are\\_unsupervised\\_multitask\\_learners.pdf](https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf).
- N. Saunshi, N. Dikkala, Z. Li, S. Kumar, and S. J. Reddi. Reasoning with latent thoughts: On the power of looped transformers, 2025. URL <https://arxiv.org/abs/2502.17416>.
- J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017. URL <http://arxiv.org/abs/1707.06347>.
- Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Q. Team. Qwen3, April 2025. URL <https://qwenlm.github.io/blog/qwen3/>.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017. URL <http://arxiv.org/abs/1706.03762>.
- J. Wei, X. Wang, D. Schuurmans, M. Bosma, E. H. Chi, Q. Le, and D. Zhou. Chain of thought prompting elicits reasoning in large language models. *CoRR*, abs/2201.11903, 2022. URL <https://arxiv.org/abs/2201.11903>.
- A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang, G. Dong, H. Wei, H. Lin, J. Tang, J. Wang, J. Yang, J. Tu, J. Zhang, J. Ma, J. Xu, J. Zhou, J. Bai, J. He, J. Lin, K. Dang, K. Lu, K. Chen, K. Yang, M. Li, M. Xue, N. Ni, P. Zhang, P. Wang, R. Peng, R. Men, R. Gao, R. Lin, S. Wang, S. Bai, S. Tan, T. Zhu, T. Li, T. Liu, W. Ge, X. Deng, X. Zhou, X. Ren, X. Zhang, X. Wei, X. Ren, Y. Fan, Y. Yao, Y. Zhang, Y. Wan, Y. Chu, Y. Liu, Z. Cui, Z. Zhang, and Z. Fan. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024a.

- A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024b.
- S. Yang, E. Gribovskaya, N. Kassner, M. Geva, and S. Riedel. Do large language models latently perform multi-hop reasoning?, 2024c. URL <https://arxiv.org/abs/2402.16837>.
- S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023. URL <https://arxiv.org/abs/2305.10601>.
- Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, T. Fan, G. Liu, L. Liu, X. Liu, H. Lin, Z. Lin, B. Ma, G. Sheng, Y. Tong, C. Zhang, M. Zhang, W. Zhang, H. Zhu, J. Zhu, J. Chen, J. Chen, C. Wang, H. Yu, W. Dai, Y. Song, X. Wei, H. Zhou, J. Liu, W.-Y. Ma, Y.-Q. Zhang, L. Yan, M. Qiao, Y. Wu, and M. Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- Y. Yue, Y. Yuan, Q. Yu, X. Zuo, R. Zhu, W. Xu, J. Chen, C. Wang, T. Fan, Z. Du, X. Wei, X. Yu, G. Liu, J. Liu, L. Liu, H. Lin, Z. Lin, B. Ma, C. Zhang, M. Zhang, W. Zhang, H. Zhu, R. Zhang, X. Liu, M. Wang, Y. Wu, and L. Yan. Vapo: Efficient and reliable reinforcement learning for advanced reasoning tasks, 2025. URL <https://arxiv.org/abs/2504.05118>.
- A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola. *Dive into Deep Learning*. Cambridge University Press, 2023. <https://D2L.ai>.

## A APPENDIX